

Digital Payment Wallet System in Java

Aim / Objective

To design and implement a Digital Payment Wallet System in Java using an interface, simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.

Description

The Digital Payment Wallet System creates a wallet by accepting user details through the keyboard and performs basic digital payment operations. The program uses an interface for payment services, encapsulation for wallet data, and custom exceptions to validate UPI IDs, transaction amounts, and wallet balance before processing a payment.

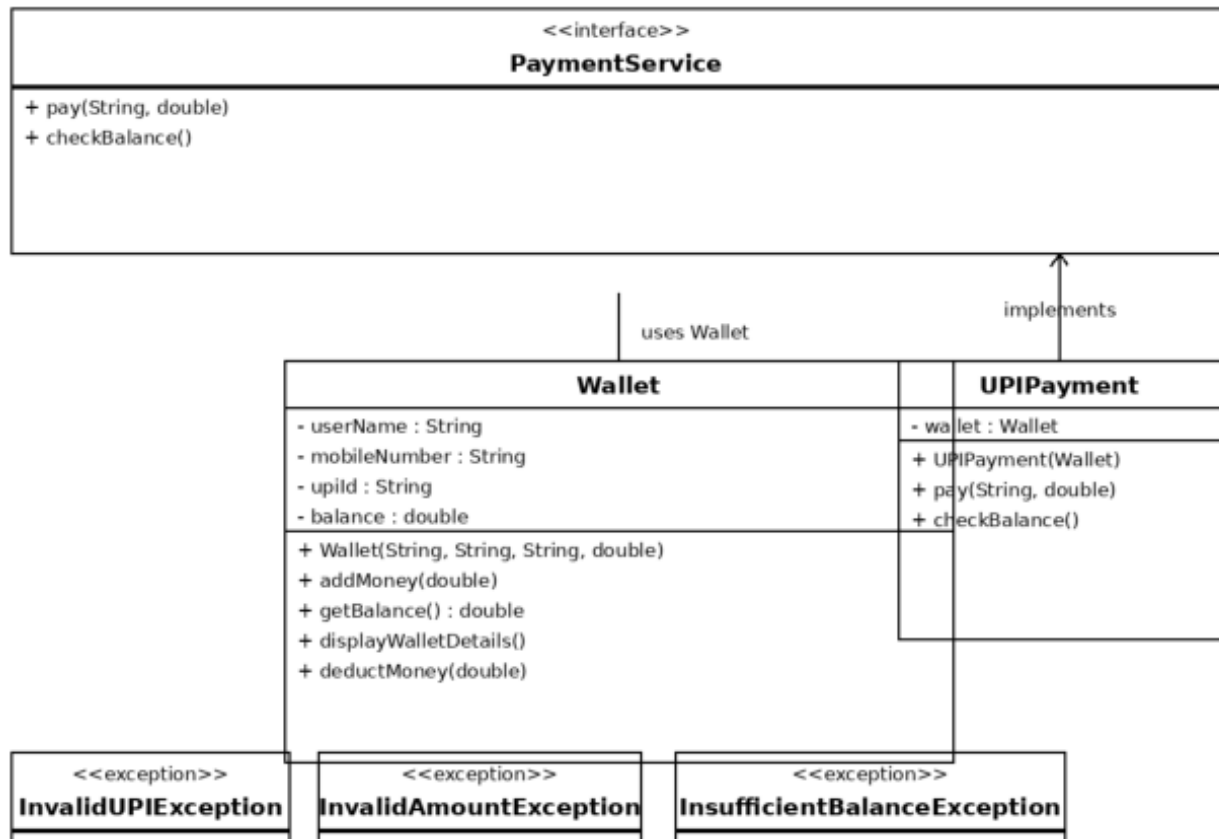
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse.

Software: Java Development Kit (JDK), Java compiler, Java Runtime Environment (JRE), and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create a Scanner object for user input.
3. Read the user's name, mobile number, UPI ID, and initial balance.
4. Create a Wallet object using the entered details.
5. Create a UPIPayment object for the wallet.
6. Display the wallet details.
7. Read the amount to add and validate it using addMoney().
8. Read the receiver UPI ID and payment amount.
9. Validate the receiver UPI ID before processing the payment.
10. If the UPI ID is invalid, throw InvalidUPIException.
11. Validate the payment amount and throw InvalidAmountException for zero or negative amounts.
12. Check the wallet balance and throw InsufficientBalanceException when the balance is insufficient.
13. If all validations are successful, deduct the payment amount and display the success message.
14. Use try-catch-finally to handle transaction exceptions.
15. Display the final wallet details and close Scanner.



Source Code

```

import java.util.Scanner;

interface PaymentService {
    void pay(String receiverUPI, double amount)
        throws InvalidUPIException, InvalidAmountException,
            InsufficientBalanceException;
    void checkBalance();
}

class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    Wallet(String userName, String mobileNumber,
        String upiId, double balance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Invalid amount");
        }
        balance = balance + amount;
        System.out.println("Money added: " + amount);
    }

    double getBalance() {
        return balance;
    }
}

```

```

void displayWalletDetails() {
    System.out.println("User Name      : " + userName);
    System.out.println("Mobile Number : " + mobileNumber);
    System.out.println("UPI ID       : " + upiId);
    System.out.println("Balance      : " + balance);
}

void deductMoney(double amount) {
    balance = balance - amount;
}
}

class UPIPayment implements PaymentService {
    private Wallet wallet;

    UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    public void pay(String receiverUPI, double amount)
        throws InvalidUPIException, InvalidAmountException,
        InsufficientBalanceException {
        if (!receiverUPI.contains("@") ||
            receiverUPI.startsWith("@") ||
            receiverUPI.endsWith("@")) {
            throw new InvalidUPIException("Invalid UPI ID");
        }

        if (amount <= 0) {
            throw new InvalidAmountException("Invalid payment amount");
        }

        if (amount > wallet.getBalance()) {
            throw new InsufficientBalanceException(
                "Insufficient wallet balance");
        }

        wallet.deductMoney(amount);
        System.out.println("Payment successful");
        System.out.println("Paid: " + amount);
        System.out.println("To: " + receiverUPI);
    }

    public void checkBalance() {
        System.out.println(
            "Available Balance: " + wallet.getBalance());
    }
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

```

```

}

class InvalidUPIException extends Exception {
    InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    InvalidAmountException(String message) {
        super(message);
    }
}

public class DigitalPaymentDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("==== DIGITAL PAYMENT WALLET ====");
        System.out.print("Enter User Name: ");
        String userName = sc.nextLine();
        System.out.print("Enter Mobile Number: ");
        String mobileNumber = sc.nextLine();
        System.out.print("Enter UPI ID: ");
        String upiId = sc.nextLine();
        System.out.print("Enter Initial Balance: ");
        double balance = sc.nextDouble();

        Wallet wallet = new Wallet(
            userName, mobileNumber, upiId, balance);
        UPIPayment payment = new UPIPayment(wallet);

        System.out.println("\n==== WALLET DETAILS ====");
        wallet.displayWalletDetails();
        System.out.println("\n==== TRANSACTION ====");

        try {
            System.out.print("Enter amount to add: ");
            double addAmount = sc.nextDouble();
            wallet.addMoney(addAmount);

            sc.nextLine();
            System.out.print("Enter Receiver UPI ID: ");
            String receiverUPI = sc.nextLine();
            System.out.print("Enter Payment Amount: ");
            double paymentAmount = sc.nextDouble();

            payment.pay(receiverUPI, paymentAmount);
            payment.checkBalance();
        } catch (InvalidUPIException |
            InvalidAmountException |
            InsufficientBalanceException e) {
            System.out.println(
                "Transaction failed: " + e.getMessage());
        } finally {
            System.out.println("Transaction process completed.");
        }

        System.out.println("\n==== FINAL WALLET ====");
        wallet.displayWalletDetails();
        sc.close();
    }
}

```

Compilation and Execution Commands

Save as: DigitalPaymentDemo.java
 Compile: javac DigitalPaymentDemo.java
 Execute: java DigitalPaymentDemo

Output Screens

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
===== TRANSACTIONS =====  
Enter amount to add: 2000  
Money added: 2000.0  
Enter Receiver UPI ID: ravi@upi  
Enter Payment Amount: 1500  
Payment successful  
Paid: 1500.0  
To: ravi@upi  
Available Balance: 5500.0  
Transaction process completed.
```

Possible Output – Invalid Transaction Handling

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Enter Receiver UPI ID: wrongupi  
Enter Payment Amount: 500  
Transaction failed: Invalid UPI ID  
Transaction process completed.
```

Result

Thus, the Digital Payment Wallet System was implemented successfully to demonstrate interfaces, encapsulation, String validation, custom exceptions, exception handling, user input, wallet transactions, and UPI payment processing using Java.